

Member ID: _____

Time: _____

Rank: _____



JAVA PROGRAMMING

(340)

NATIONAL 2023

Production (90 minutes)

Crypto Project Transfer

_____ (460 points)

Industry Certification (50 minutes)

IT Specialist Java

GENERAL GUIDELINES:

Failure to adhere to any of the following rules will result in disqualification:

1. No equipment, supplies, or materials other than those specified for this event are allowed in the testing area. No previous BPA tests and/or sample tests (handwritten, photocopied, or keyed) are allowed in the testing area.
2. Electronic devices will be monitored according to ACT standards.
3. Your name and/or school name should *not* appear on work you submit for grading.
4. Ensure that the files required to run your program are present and will execute.
5. Once you have complete problem listed below, upload all competition materials (including project files, etc.) to the National BPA Pre-submit System at presubmit.bpa.org.
6. Compress all files in a single archive (zip file) and use the following naming conventions for your documents:

JAVA-MemberID.zip

Assumptions to make when taking this assessment:

- Your program will be reading from three different text files that will make a specific object based upon that file (3 different types of objects). You will need to consolidate the three different objects into a generic wrapper class that will be used to write all three files into a single Portfolio.csv file and print the exact same results to the console.
- The program will automatically exit after the txt files have been read, required objects created, and the identical formatted output is produced in the console and Portfolio.csv file.
- All txt files for creating the objects in question have been provided. Given the complexity of the project, you will have access to a single file that will contain all of the classes needed to complete the task. It is not necessary to store object classes as their own file; however, if you choose to do so, there is no penalty.
- The output in console needs to be as close as possible to the example provided. Minor deviations that have no impact on the output formatting, calculations, or reading of the txt files will have no penalty. However, your console output and the Portfolio.csv output need to match.
- The text files should be read in this order: Coins.txt, Tokens.txt, NFT.txt

Development Standards:

- Your Code must use a consistent variable naming convention.
- All subroutines (if any), functions (if any), and methods (if any) must be documented with comments explaining the purpose of the method, the input parameters (if any), and the output (if any). Readability is the goal of any code you write.
- If you create a class, then you must use Javadoc comments.

Crypto Project Transfer

Like other financial markets, the cryptocurrency marketplace is no different in that it offers a multitude of various products that can be purchased by investors. For simplicity purposes in this particular program we're going to concentrate on 3 product types which are coins, tokens, and NFT's. When purchasing these various investment vehicles (fancy way of saying investment products) the user will most likely encounter different computer systems. These computer systems sometimes create complex proprietary, or simple generic files that will allow information to be transferred back into centralized systems to better manage all of the data for each individual client and all of the other support software that's involved in these processes.

In this challenge we will focus on generic text files which you should be very familiar with by now. You will be charged with taking three different text files that each support a different investment vehicle as stated previously. Your program will import each text file and store it and its associated object class. For example, your program will read the Coins text file and convert each record into a coin object. Next your program will extract the data from a Tokens text file and store those records as token objects. Third your program will read an NFT text file and store each of those records as an NFT object.

Once the three text files have been read and the records have been created into their appropriate objects, you will be required to consolidate all the records from the three different types of

objects and write them to a new Portfolio.csv file that can be used to store the information into a centralized database. We are not going to concentrate on the processes that would entail storing the information into a database. Instead, you'll be focusing upon taking the three different types of objects and putting them into a generic data structure that will allow you to systematically create one single file from one location within your program. In addition, you will also be challenged with adding a timestamp to the output. Your output will be printed to the console, and it should look exactly like as to what is printed, or written, to the new Portfolio.csv file that you will be creating. NOTE: the Portfolio.csv file when opened is most likely going to be opened into a spreadsheet; do not be concerned with this, it is to be expected. The exactness of the appearance between the two will be measured by their organization of fields and records.

The major stakeholder of this project is preparing for the explosive growth of the cryptocurrency marketplace and its ever-expanding investment vehicles. To help prepare for this growth that will be reflected in the software systems being developed, the classes that you create within the program will be utilizing interfaces that will need to be implemented properly. The information for the interfaces will be provided to you and you will not be required to create the interfaces, only implement them. You will also be challenged with having to create and use inheritance from a parent class for the investment vehicles.

Input

In this program there will be no manual user input; instead, the input will come from using the BufferedReader to extract information through the text file and separate the fields of the records based upon the delimiter (in this program it is a semicolon). The methods to read the file and import its data is already created for the student. The program will be required to alter the code so that it reads the three separate text files in order (Coins.txt, Tokens.txt, NFT.txt) and create required objects for each of the different types, and the incorporate them into a central wrapper class object (hint: a generic data structure).

Output

The output for this challenge will require that identical information (order and format) is written to an external Portfolio.csv file called "Portfolio" (Portfolio.csv) that can be imported into a database; additionally, the output will also be printed to the console. A sample is provided in the Sample Output section. NOTE: There is a possibility that the text size and font type will be different for the console and the Portfolio.csv file: this is fine. There is also a possibility your Portfolio.csv file will open in a spreadsheet: this is totally expected and acceptable. However, it

is imperative that the order of the fields (along with field headers) for each record, and the order of the records match the criteria exactly. Furthermore, at the top of the Portfolio.csv file and at the top of the console output, there should be require a timestamp with the day, date, time, and year. The format for the timestamp's spacing and additional characters to create readability can vary, however all four fields of the timestamp need to be present in the exact order. When graders run your program, the timestamp will need to be current, in other words you need to extract the timestamp information from your system...do not manually hardcode the timestamp. It should also be noted that the quantitative information for the output will not need to be formatted (commas, decimal places, monetary markings, etc.) other than ensuring that it is correctly extracted from the text files intact and in the proper order. If you choose to format the numbers it will not be penalized, but it will NOT earn any points.

Infrastructure Requirements

The building of the program also requires that certain classes are required to use an interface. There are three classes that will each use a specific interface. In addition, the Coins, Tokens, and NFTs must inherit their behaviors and attributes from the parent class InvestmentProducts.

Sample Output

The following is the exact order that should be printed to the console and to the Portfolio.csv file. NOTE: the formatting of the font type and size, as well as spacing may differ. However, the order of the information needs to be exactly like the example below: Date, Coins (entries 1-4), Tokens (entries 5-8), NFTs (entries 9-12). Any scientific notation created by Java will be ignored (i.e. 5.12E-4). Also notice in the output that it begins with the name of the investment; in the imported text files (not shown) each record will begin with a symbol (this needs to be removed for the output). Abbreviation Tips: Symbol (Sym); Quantity (Qty); Current Price (CP); Average Price (AP); Buy Price (BP).

```
*****
*****
Wed Dec 29 09:51:37 CST 2021
*****

Name: Bitcoin, Sym: BTC, Qty: 5.12E-4, CP: 48912.0, AP: 56234.0, BP: 59343.0
Name: Ethereum, Sym: ETH, Qty: 2.345, CP: 4323.0, AP: 4465.0, BP: 3738.0
Name: Cardano, Sym: ADA, Qty: 3245.0, CP: 1.34, AP: 2.21, BP: 2.09
Name: Dogecoin, Sym: DOGE, Qty: 33922.0, CP: 0.1595, AP: 0.2343, BP: 0.2386
Name: Polygon, Sym: MATIC, Qty: 56.34, CP: 2.34, AP: 1.84, BP: 0.34
Name: Sandbox, Sym: SAND, Qty: 1222.0, CP: 5.87, AP: 2.77, BP: 2.72
Name: Shiba-Inu, Sym: SHIB, Qty: 1345987.0, CP: 3.395E-5, AP: 5.645E-5, BP:
2.35E-5
Name: Uniswap, Sym: UNI, Qty: 234.2, CP: 14.75, AP: 11.23, BP: 5.82
Name: Ethicorns, Sym: BNB, Qty: 6.334, CP: 5.34, AP: 4.34, BP: 5.67
Name: ChainSports, Sym: ETH, Qty: 0.2, CP: 1.2, AP: 2.2, BP: 0.3
Name: MetaLands, Sym: MATIC, Qty: 1.645, CP: 0.345, AP: 3.2123, BP: 1.4
Name: League of Heroes, Sym: MATIC, Qty: 0.45, CP: 4.2, AP: 2.12, BP: 2.98
```

Sample Run Hint #1: The information in the text files have spaces within the entries that are between the delimiters. When the program reads this information, it will transfer the spaces back to the program. If the spaces are not removed, this could have detrimental effects on your ability to complete all of the required tasks, especially when you are working with quantitative values.

Sample Run Hint #2: When selecting a generic data structure, be sure to pick one that is flexible and will allow you to utilize FIFO.

Sample Run Hint #3: Remember that the files need to read in the following order: Coins, Tokens, and NFTs.

Sample Run Hint #4: Use the type symbol at the beginning of the text files to have the program determine which object to create (C = Coin, T = Token, N = NFT).

Requirements

1. You will be required to create multiple classes to complete this project. Certain classes have already been created for you that are based upon a predetermined solution. You are free to alter these classes as you see fit; however, you will still be required to complete the portions of the rubric to earn points.
2. The class that hosts the Main method will be called **CryptoProjectTransfer**.
3. The object class **InvestmentProducts** must use the interface called **Assett_Interface**; this class already has data fields created to assist you.
4. **Coins**, **Tokens**, and **NFT** are children classes of the **InvestmentProducts** class. These children classes must be used to create the objects that are read from the text files.
5. The program must use the **toString()** method to retrieve data from **Coins**, **Tokens**, and **NFT** objects.
6. The **MyWrapper** class must use the interface called **Wrapper_Interface**. One of the required methods of this class is called **fileWriter()**; this method will be used to create the consolidated “Portfolio.csv” file of the three investment types. Another method required is called **exportDate(PrintWriter out)**; this method will be used to create the date and then place that date into the newly created text field as well as print it out to console with the rest of the records read from the investment type text files. This class also has another required method called **openWriter(String name)**; this method is used to create a **PrintWriter** object that will allow you to create a new file to write the consolidated investment information.
7. The **RecordReader** class must use the interface called **FileReader_Interface**. This class has a completed method that is required **getReader(String name)**; this method returns the **BufferedReader** object that is needed to read the text files. In addition, the method **readBlock(BufferedReader in)** has been partially created for you. The missing portion requires you to check the type of investment, choose the proper constructor and return the object (hint look at the method signature and notice it is returning a generic Object). The final required method **inputFileReader()**; it is recommended that you should use this method to call **readBlock** method and loop through all of the text files and figure out a way to get it back to **MyWrapper** class data structure.
8. Your outputted file needs to be called “Portfolio.csv” and written to the same folder as the driver class for the program.
9. Your contestant number must appear as a comment at the top of the main source code file.
10. The program will display the outputs as shown in the example on the previous page.

Solution and Project		
The project is present on the flash drive	_____	10 points
The projects main class is named CryptoProjectTransfer	_____	10 points
The parent class of Coins, Tokens, NFT classes is InvestmentProducts (10 pts each)	_____	30 points
InvestmentProducts class properly applies the Assett_Interface	_____	20 points
MyWrapper class properly applies the Wrapper_Interface	_____	20 points
RecordReader class properly applies the FileReader_Interface	_____	20 points
Program Execution		
The program runs from the USB flash drive	_____	10 points
<i>If the program does not execute, then the remaining items in this section receive a score of zero.</i>		
The final printout to console and the text file match (excluding font size and types); this includes abbreviations, colons, commas, and data (1 error = no points)	_____	20 points
The order of the fields in the records printed matches the proper order: (1) Name; (2) Symbol; (3) Quantity; (4) Current Price; (5) Avg. Price; (6) Buy Price	_____	20 points
The order of the records printed matches the following: (1) Coins; (2) Tokens; (3) NFT	_____	20 points
The final outputted file is called “Portfolio.csv” and is stored in the same folder as the driver class.	_____	10 points
Source Code Review		
The source code is properly commented	_____	10 points
A comment containing the contestant number is present at the top of the program	_____	10 points
InvestmentProducts class uses the Assett_Interface required methods; you will not be penalized for bypassing them and creating alternative solutions	_____	30 points
Coins, Tokens, and NFT class constructors call super () with parameters to be created with InvestmentProducts class (10 points each)	_____	30 points
InvestmentProducts class toString() method is used to print all detail for its objects.	_____	10 points
MyWrapper class uses the Wrapper_Interface required methods; you will not be penalized for bypassing them and creating alternative solutions	_____	30 points
MyWrapper class contains a data structure that can host all three types of investment products	_____	20 points

The fileWriter() method in MyWrapper class is used to write the records to the final “Portfolio.csv” file.		30 points
The exportDate(PrintWriter out) method in MyWrapper class is used to create the date time stamp from the system clock for the file and the print to console.		20 points
The openWriter(String name) method in MyWrapper class is used create the Portfolio.csv file		30 points
RecordReader class uses the FileReader_Interface required methods; you will not be penalized for bypassing them and creating alternative solutions		30 points
readBlock(BufferedReader in) method in the RecordReader class reads and returns the proper investment object type.		20 points
Total Points		/460 points

Files and Class Behaviors/Attributes: Help Information For Items Given

Files Needed (4 total and all are PROVIDED):

- CryptoProjectTransfer.java (contains all of the classes)
- Text files: Coins.txt; Tokens.txt, NFT.txt

NOTE: you will program methods in all of the classes. Very little has been provided other than items that should be simple to create, and they are only given to help save time.

public class CryptoProjectTransfer Behaviors and Attributes:

- public static void main (String args []) PROVIDED
- ACTION: there is nothing else in this class and it drives the program

class InvestmentProducts:

- Attributes PROVIDED
 - private String type: what type of product it is (C: coin; T: token; N: NFT)
 - private String name: name of the investment(i.e. Bitcoin)
 - private String symbol: all CAPS abbreviation/representation of the title (.i.e. BTC)
 - private double quantity: a rational number of how much is own
 - private double currentPrice: value of investment (this will not be formatted)
 - private double avgPrice: average price of investment (this will not be formatted)
 - private double buyPrice: how much was paid for investment (this will not be formatted)
- Constructors PROVIDED: two are provided to help save time typing. Either can be used. See comments.
- interface Assett_Interface (ACTION: see interface): includes standard getter and setter methods for the private variables, as well as the requirement to use toString().

classes Coins, Tokens, NFT (ACTION: nothing is provided because they are children of InvestmentProducts and have identical requirements): These should be very easy to implement.

class MyWrapper (ACTION: nothing is provided):

- interface Wrapper_Interface: includes three methods that will help in creating the output for the file and console print.

class RecordReader (PARTIAL: some methods and variables have been provided):

- Attribute: private BufferedReader in: use this for the getReader method
- public Object readBlock(BufferedReader in): this has been partially created for you. The code needed is the conditional check to see what type of investment product it is and perform the appropriate operations. Hint, you are returning a generic object.
- interface FileReader_Interface: see the descriptions below.

NOTE: all of the interface descriptions below are suggestions on how they should be used, you can use the methods as needed; however, the interface as it is created will required to be used in its original format. You are not allowed to alter the interfaces.

interface Assett_Interface: these are common getter and setter methods for variables. Take note that the toString() method is included in this list of required methods.

interface Wrapper_Interface:

- public void exportDate(PrintWriter out): the “out” variable will have been created the openWriter() method; it is the file you will be writing the date to. Note: you can use this same date for the console print, too
- public PrintWriter openWriter(String w): this will create the file based upon the string name passed to it, which should be Porfolio.csv
- public void fileWriter(): use this method to write the investment records to the newly created file

interface FileReader_Interface:

- public void inputFileReader(): use this method to drive the reading of the files through the readBlock method. Hint: you will probably need some form of loop to extract the data from the files.
- public Object readBlock(BufferedReader in): this has been partially created for you. The code needed is the conditional check to see what type of investment product it is and perform the appropriate operations. Hint, you are returning a generic object.
- public BufferedReader getReader(String name): provided for you, this will allow the program to read the given file (“Coins.txt”); this could be one of the first methods to be used since everything else depends on being able to read data from the text file.